קטע קוד A

```
1. struct BaseExpression {
2.   virtual ~BaseExpression() {}
3.   virtual double eval() const = 0;
4.   virtual ostream& print(ostream& out) const = 0;
5. };
6.
7. class Expression {
8.   const BaseExpression* const e;
9. public:
10.   Expression(BaseExpression* e): e (e) {}
11.   Expression(const Expression& e) = delete;
12.   ~Expression() {
13.       // cout << "about to delete " << *this << endl;
14.       delete e;
15.   }
16.   double eval() const { return e->eval(); }
17.   friend ostream& operator<<(ostream& out, const Expression& e) {
18.     return e.e->print(out);
19.   }
20. };
21.
22. class Number: public BaseExpression {
23.   double value;
24. public:
25.   Number(double d) : value(d) {}
26.   double eval() const override { return value; }
27.   ostream& print(ostream& out) const override {
28.     return out << value;
29.   }
30. };
31.
32. template<char Sign>
33. class BinaryExpression : public BaseExpression {
34.   Expression e1, e2;
35.   virtual double evalImpl(double d1, double d2) const = 0;
36.   BaseExpression* to_expression(double num) const {
37.       return new Number(num);
38.   }
39.   BaseExpression* to_expression(BaseExpression* e) const {
40.       return e;
41.   }
42. public:
43.   template<typename E1, typename E2>
44.   BinaryExpression (E1 e1, E2 e2)
45.       : e1(to_expression(e1)), e2(to_expression(e2)) {}
46.   double eval() const override {
47.     return evalImpl(e1.eval(), e2.eval());
48.   }
```

```
49.   ostream& print(ostream& out) const override {
50.     return out << '(' << e1 << ' ' << Sign << ' ' << e2 << ')';
51.   }
52. };
53.
54. class Sum: public BinaryExpression<'+'> {
55.   double evalImpl(double d1, double d2) const override {
56.     return d1 + d2;
57.   }
58. public:
59.   using BinaryExpression::BinaryExpression; // inheriting base ctor
60. };
61.
62. class Minus: public BinaryExpression<'-'> {
63.   double evalImpl(double d1, double d2) const override {
64.     return d1 - d2;
65.   }
66. public:
67.   using BinaryExpression::BinaryExpression; // inheriting base ctor
68. };
69.
70. class Mult: public BinaryExpression<'*'> {
71.   double evalImpl(double d1, double d2) const override {
72.     return d1 * d2;
73.   }
74. public:
75.   using BinaryExpression::BinaryExpression; // inheriting base ctor
76. };
77.
78. class Div: public BinaryExpression<'/'> {
79.   double evalImpl(double d1, double d2) const override {
80.     return d1 / d2;
81.   }
82. public:
83.   using BinaryExpression::BinaryExpression; // inheriting base ctor
84. };
```

להלן מספר main אפשריים:

A1 main

```
1. int main() {
2.   Expression e = new Sum(new Minus(10, 2), -2);
3.   std::cout << e << " = " << e.eval() << std::endl;
4. }
```

A2 main

```
1. int main() {
2.   Expression e = new Mult(
3.       new Div( new Number(10), new Number(-2) ),
4.       new Number(-2) );
5.    std::cout << e.eval() << std::endl;
6. }
```

A3 main

```
1. int main() {
2.   auto e = new Mult(10, 2);
3.   std::cout << e->eval() << std::endl;
4.   delete e;
5. }
```

A4 main

```
1. int main() {
2.   auto e1 = new Sum(1, 3);
3.   auto e2 = new Div( e1, 2 );
4.   std::cout << e2->eval() << std::endl;
5.   delete e2;
6. }
```

קטע קוד B

```
 1. // strings, counted in a map
 2. using Groups = map<std::string, size_t>;
 3.
 4. class MyString {
 5.   static Groups groups;
 6.   list<Groups::iterator> substrings;
 7.   static Groups::iterator joinGroup(const std::string& s) {
 8.     auto itr = groups.find(s);
 9.     if(itr == groups.end()) {
10.       // see explanation regarding map::insert at line 68
11.       itr = groups.insert({s, 1}).first;
12.     }
13.     else {
14.       ++(itr->second);
15.     }
16.     return itr;
17.   }
18.   static void leaveGroups(const list<Groups::iterator>& subs) {
19.     for(auto itr: subs)
20.       if(--(itr->second) == 0) groups.erase(itr);
21.   }
22.   static void addOneToAll(const list<Groups::iterator>& subs) {
23.     for(auto itr: subs) {
24.       ++(itr->second);
25.     }
26.   }
27. public:
28.   static void printGroups() {
29.     for(auto g : groups) {
30.       cout << '\"' << g.first << "\" (" << g.second << ")\n";
31.     }
32.   }
33.   MyString(const char* s = "") {
34.     substrings.push_back(joinGroup(s));
35.   }
36.   ~MyString() { leaveGroups(substrings); }
37.   MyString(const MyString& s): substrings(s.substrings) {
38.     addOneToAll(substrings);
39.   }
40.   MyString& operator=(const MyString& s) {
41.     if(this != &s) {
42.         leaveGroups(substrings);
43.         substrings = s.substrings;
44.         addOneToAll(substrings);
45.     }
46.     return *this;
47.   }
```

```
48.   MyString& operator+=(MyString s) {
49.     // the call below takes (and removes)
50.     // all elements in s.substrings
51.     // and adds them to the end of this->substrings
52.     substrings.splice(
53.       /*insert to my end:*/ substrings.end(),
54.       /*elements from:*/ s.substrings,
55.       /*in the range:*/ s.substrings.begin(), s.substrings.end());
56.     return *this;
57.   }
58.   MyString operator+(const MyString& s) const {
59.     MyString result = *this;
60.     return result += s;
61.   }
62.   friend ostream& operator<<(ostream& out, const MyString& s) {
63.     for(auto itr: s.substrings) out << itr->first;
64.     return out;
65.   }
66. };
67.
68. // in MyString.cpp
69. Groups MyString::groups;
70.
71. // explanation regarding map::insert (line 11):
72. // map.insert returns std::pair<iterator, bool>
73. // 'first' is an iterator pointing to the new item or to
74. // an item with the same key that already existed, which
75. // prevented the insertion
76. // 'second' is whether insertion was performed
77.
78. int main() {
79.   MyString s1 = "hi";
80.   MyString s2 = s1;
81.   MyString* s3 = new MyString(s2);
82.   // s2 += " you";
83.   // cout << s2 + " you" << endl;
84.   MyString::printGroups();
85.   // delete s3;
86.   // s3 = &s1;
87.   s1 = "!!!";
88.   (*s3) += s1;
89.   cout << *s3 << endl;
90.   MyString::printGroups();
91.   delete s3;
92. }
```

קטע קוד C – לשאלות הבונוס

```
 1. struct Expression {
 2.   virtual ~Expression() {}
 3.   virtual double eval() const = 0;
 4.   virtual ostream& print(ostream& out) const = 0;
 5.   virtual Expression* clone() const & = 0;
 6.   virtual Expression* clone() && = 0;
 7.   friend ostream& operator<<(ostream& out, const Expression& e) {
 8.     return e.print(out);
 9.   }
10. };
11.
12. template<typename ActualType>
13. class MultiExpression : public Expression {
14.
15.   std::vector<Expression*> subexpressions;
16.
17.   virtual double evalImpl(double d1, double d2) const = 0;
18.   MultiExpression (const std::vector<Expression*>& vec) {
19.     for(auto p: vec) subexpressions.push_back(p->clone());
20.   }
21.   MultiExpression (std::vector<Expression*>&& vec)
22.     : subexpressions(std::move(vec)) {}
23.   void free() {
24.     for_each(subexpressions.begin(),subexpressions.end(),
25.              [](Expression* e) {delete e;} );
26.   }
27. public:
28.   template<typename... Ts>
29.   MultiExpression (Ts&&... e)
30.     : subexpressions({std::forward<Ts>(e).clone()...}) {}
31.   ~MultiExpression() override { free(); }
32.   MultiExpression(const MultiExpression& other)
33.     : MultiExpression(other.subexpressions) {}
34.   MultiExpression(MultiExpression&& other)
35.     : MultiExpression(std::move(other.subexpressions)) {}
36.
37.   MultiExpression& operator=(MultiExpression&& other) {
38.     free();
39.     subexpressions = std::move(other.subexpressions);
40.     return *this;
41.   }
42.
43.   MultiExpression* clone() const & override {
44.     return new ActualType(subexpressions);
45.   }
46.   MultiExpression* clone() && override {
47.     return new ActualType(std::move(subexpressions));
48.   }
```

```
49.   double eval() const override {
50.     double head = subexpressions[0]->eval();
51.     for_each(subexpressions.begin() + 1, subexpressions.end(),
52.         [&head, this](Expression* e) {
53.                 head = evalImpl(head, e->eval()); } );
54.     return head;
55.   }
56.   ostream& print(ostream& out) const override {
57.     out << '(' << *subexpressions[0];
58.     for_each(subexpressions.begin() + 1,subexpressions.end(),
59.         [&out, this](const Expression* e) {
60.               out << ' ' << ActualType::Sign << ' ' << *e; } );
61.     return out << ')';
62.   }
63. };
64.
65. class Number: public Expression {
66.   double value;
67. public:
68.   Number(double d) : value(d) {}
69.   Number* clone() const & override { return new Number(value); }
70.   Number* clone() && override { return new Number(value); }
71.   double eval() const override { return value; }
72.   ostream& print(ostream& out) const override {
73.     return out << value;
74.   }
75. };
76.
77. class Sum: public MultiExpression<Sum> {
78.   double evalImpl(double d1, double d2) const override {
79.     return d1 + d2;
80.   }
81. public:
82.   static constexpr char Sign = '+';
83.   using MultiExpression::MultiExpression;
84. };
85.
86. int main() {
87.     auto e1 = Sum( Number(-2), Number(5) );
88.     auto e2 = Sum( std::move(e1), Sum( Number(1), Number(2) ) );
89.     std::cout << e2 << " = " << e2.eval() << std::endl;
90.     // std::cout << e1.eval() << std::endl;
91.     auto e3 = Sum( e2, e2 );
92.     std::cout << e3.eval() << std::endl;
93.     // e1 = Sum( Number(9), Number(9) );
94.     // std::cout << e1.eval() << std::endl;
95. }
```

בהצלחה !